

Repetitive motion of redundant robots planned by three kinds of recurrent neural networks and illustrated with a four-link planar manipulator's straight-line example

Yunong Zhang*, Zhiguo Tan, Ke Chen, Zhi Yang, Xuanjiao Lv

School of Information Science and Technology, Sun Yat-Sen University, Guangzhou 510275, China

ARTICLE INFO

Article history:

Received 2 December 2007

Received in revised form

8 December 2008

Accepted 14 January 2009

Available online 24 January 2009

Keywords:

Neural networks

Four-link planar robot manipulator

Joint physical limits

Drift-free criterion

Quadratic performance index

ABSTRACT

In this paper, a dual neural network, LVI (linear variational inequalities)-based primal-dual neural network and simplified LVI-based primal-dual neural network are presented for online repetitive motion planning (RMP) of redundant robot manipulators (with a four-link planar manipulator as an example). To do this, a drift-free criterion is exploited in the form of a quadratic performance index. In addition, the repetitive-motion-planning scheme could incorporate the joint physical limits such as joint limits and joint velocity limits simultaneously. Such a scheme is finally reformulated as a quadratic program (QP). As QP real-time solvers, the aforementioned three kinds of neural networks all have piecewise-linear dynamics and could globally exponentially converge to the optimal solution of strictly-convex quadratic-programs. Furthermore, the neural-network based RMP scheme is simulated based on a four-link planar robot manipulator. Computer-simulation results substantiate the theoretical analysis and also show the effective remedy of the joint angle drift problem of robot manipulators.

© 2009 Elsevier B.V. All rights reserved.

1. Introduction

Kinematically redundant manipulators are those having more degrees of freedom (DOF) than required for position and orientation [1]. Redundant degrees of freedom make it possible to achieve some useful objectives such as collision avoidance with obstacles in the operational space, joint limit avoidance and/or avoiding singular configurations when the manipulator moves [2]. Since redundant manipulators have more DOF than necessary for a specific end-effector position-and-orientation task, multiple solutions exist. On the other hand, the redundancy of inverse-kinematic mappings complicates the manipulator control problem considerably, in addition to its nonlinearity. If the workspace path of an end-effector is already given (at least at that time instant t), the online computation of corresponding feasible joint trajectories for a redundant manipulator is referred to as redundancy resolution [3].

To take full advantage of the redundancy, various computational schemes have been developed. Conventionally, the general solution of redundancy problem is obtained by the pseudoinverse formulation, being one minimum-norm particular solution plus a homogeneous solution [4,5]. Most of current researchers have applied this pseudoinverse-type technique to formulate and resolve

the redundancy by considering different optimization criteria, such as minimum joint-velocity norm, singularity avoidance, obstacle avoidance and task priority control. However, among those techniques, the physical limits (e.g., joint limits and joint velocity limits) are usually not taken into account, as these inequality constraints are not easy to handle [6]. If these physical limits are not considered, a saturation may occur. As a result, the tracking error may increase, not to mention the physical damage possibly caused when a commanded joint or joint velocity hits its physical bound.

For the joint-constrained inverse kinematics, numerical algorithms of redundancy resolution have been developed with physical constraints included. For example, the Jacobian matrix was augmented in [7] by incorporating the joint limits and joint velocity limits, which requires a routine predicting which limits might be violated. The joint limits and joint velocity limits were also taken into consideration in [8]. By applying the Gram–Schmidt orthogonalization procedure, a formulation was developed to express the general solution in terms of the redundant joint velocities only. Cheng et al. [6] formulated the constrained kinematic redundancy problem into a quadratic programming (QP) form. A compact QP method, using Gaussian elimination with partial pivoting, was finally developed in [9] to improve the computational efficiency. However, the numerical solutions to redundant inverse-kinematics are in general computationally intensive. With more physical limits and additional tasks considered, the computational process entails more time that may hinder the online, high-precision and/or real-time robotic applications, especially in high-DOF sensor-based manipulator systems.

* Corresponding author. Tel.: +86 20 84113597; fax: +86 20 84113572.

E-mail address: ynzhang@ieee.org (Y. Zhang).

URL: <http://www.ee.sysu.edu.cn/teacher/detail.asp?sn=129> (Y. Zhang).

In recent years, as parallel distributed computational models, neural networks have been developed for the redundancy resolution of robot manipulators, e.g., [10–14]. In particular, two neural networks, namely the pseudoinverse network [7] and the linear-programming neural network [15], were applied to the minimum infinity-norm kinematic control in [10]. The solution achieved therein explicitly minimizes joint velocities in the minimum infinity-norm sense. A two-layered primal-dual neural network was presented in [12] to online minimize the weighted joint velocity. To reduce network complexity and increase computational efficiency, an early dual-neural-network model [14] was then proposed for kinematic control of redundant manipulators. In the aforementioned schemes, it is assumed implicitly that there exists no joint limits or no joint velocity limits when solving the redundancy resolution problem. But physical limits do exist in almost any robotic systems. When a solution exceeds such mechanical joint limits or joint velocity limits (and then locks there), the desired end-effector motion-path and task may become impossible to accomplish.

In this paper, we pay attention to the design and analysis of a general approach to the repetitive motion planning (RMP) of redundant robot arms.

- (1) Firstly (in the ensuing Section 2), a drift-free criterion in the form of a quadratic function is exploited in the form of a quadratic performance index. The repetitive-motion-planning scheme, incorporating joint limits and joint velocity limits, is reformulated as a quadratic program.
- (2) Secondly (in Section 3), we present a dual neural network, which is proven to achieve global convergence. In addition, we present a primal-dual neural network based on linear variational inequalities (LVI), and its simplified model, which improve further the computational efficiency while reduce further the implementation complexity.
- (3) Thirdly (in Section 4), the neural QP-based resolution scheme is applied based on a four-link planar robot arm. Computer-simulation results demonstrate the effectiveness and efficiency of this repetitive-motion-planning scheme.

Section 5 will conclude this paper with some final remarks.

Before ending this introductory section, it is worth pointing out the main contribution of our paper as below.

- (1) The repetitive-motion-planning schemes, aiming at physically-constrained redundant robot manipulators, could be reformulated eventually as a quadratic program subject to hybrid constraints. By using the QP-based RMP scheme with joint physical limits considered, the joint angle drift problem could be remedied effectively.
- (2) Moreover, three kinds of recurrent neural networks are investigated comparatively for online QP solving as well as robots' online repetitive motion planning. Computer-simulation results via a four-link planar manipulator performing a straight-line end-effector task substantiate the efficacy of the QP-based neural-network RMP scheme. To our best knowledge, there are few papers presenting the comparative results about these three neural networks and their RMP performance.

2. Problem formulation

The end-effector position-and-orientation vector $r(t) \in R^m$ in Cartesian space and the joint-space vector $\theta(t) \in R^n$ are related nonlinearly by the forward-kinematic equation:

$$r = f(\theta). \quad (1)$$

However, Eq. (1) is usually difficult to solve due to its nonlinearity and redundancy. Such a problem solving is then considered at the velocity level:

$$J(\theta)\dot{\theta} = \dot{r}, \quad (2)$$

where $\dot{r}$ and $\dot{\theta}$ denote the m -dimensional end-effector velocity vector and the n -dimensional joint velocity vector, respectively. Jacobian matrix $J(\theta)$ is thus defined as $J(\theta) = \partial f(\theta)/\partial \theta \in R^{m \times n}$.

For redundant manipulators, because $m < n$, Eqs. (1) and (2) are under-determined and thus admit an infinite number of solutions. The conventional pseudoinverse-type solution to (2) is generally given as

$$\dot{\theta} = G\dot{r} + (I - GJ)z, \quad (3)$$

where $G \in R^{n \times m}$ denotes the pseudoinverse of J , and $z \in R^n$ is an arbitrary vector usually selected by using some of the aforementioned optimization criteria.

2.1. Drift-free inverse kinematics

It is shown in [16] that the pseudoinverse-type solutions such as (3) may not be repeatable. In other words, a closed path of the end-effector may not yield a closed path in the joint space. Such a so-called joint-angle-drift phenomenon is evidently undesirable for cyclic motion control. Certainly, we can readjust the manipulator's configuration via manipulator self-motion (i.e., to move joints but without moving the end-effector); however, this would be of less efficiency. To make the kinematic control repeatable, the minimization of the joint displacement between current state and initial state was investigated in [6,17,18]. In the problem formulation, the performance index is

$$\frac{1}{2}(\dot{\theta} + z)^T(\dot{\theta} + z) \quad \text{with } z = \lambda(\theta - \theta(0)),$$

where λ is a positive design-parameter used to scale the magnitude of the manipulator response to such joint displacements. Because almost all manipulators are physically constrained by their joint limits and joint velocity limits, it is more realistic and useful for us to consider the avoidance of such limits. Hence, we could have the following problem formulation for repetitive motion planning of redundant robot manipulators:

$$\text{minimize} \quad \frac{1}{2}(\dot{\theta} + z)^T(\dot{\theta} + z), \quad \text{with } z = \lambda(\theta - \theta(0)),$$

$$\text{subject to} \quad J(\theta)\dot{\theta} = \dot{r},$$

$$\theta^- \leq \theta \leq \theta^+, \quad (4)$$

$$\dot{\theta}^- \leq \dot{\theta} \leq \dot{\theta}^+, \quad (5)$$

where superscripts $+$ and $-$ denote the upper and lower physical limits of a joint variable vector, respectively. In addition, it is known that if the Jacobian matrix $J(\theta)$ in Eq. (2) is rank-deficient, the pseudoinverse-based methods such as (3) may encounter the kinematic-singularity problem. However, our QP-based RMP scheme could avoid such a singularity problem owing to its inverse-free processing manner (i.e., there is no explicit need to invert J in our neural-network method, which is illustrated in the ensuing sections), provided that there exists an optimal solution to such a QP scheme originally formulated around (4) and (5). Furthermore, additional constraints such as joint acceleration limits, obstacle-avoidance and singularity-avoidance constraints might also be considered in the generalized QP-based scheme (which will be one of our future research topics).

2.2. Joint limits conversion

As the redundancy is resolved at the joint-velocity level (i.e., in terms of $\dot{\theta}$), the limited joint range $[\theta^-, \theta^+]$ in (4) has to be converted into a $\dot{\theta}$ -based expression [15,19–22]. For example, we can exploit the following transformation from θ to $\dot{\theta}$ expressions (i.e., using a dynamic bound constraint):

$$\mu(\theta^- - \theta) \leq \dot{\theta} \leq \mu(\theta^+ - \theta), \quad (6)$$

where the intensity coefficient $\mu > 0$ (e.g., being 4 in our ensuing computer-simulations) is used to scale the feasible region of $\dot{\theta}$ caused by the above transformation. The choice of coefficient μ should make sure that the feasible region of $\dot{\theta}$ converted by joint limits (4) is usually not less than the original one made by joint velocity limits (5). In mathematics, $\mu \geq \max_{1 \leq i \leq n} \{(\dot{\theta}_i^+ - \dot{\theta}_i^-)/(\theta_i^+ - \theta_i^-)\}$. Note that large values of μ may cause quick joint deceleration when the manipulator approaches its joint limits. The physical meaning of this transformation is investigated in the following remark.

Remark 1. As compared to the joint-level physical limits $[\theta^-, \theta^+]$, the transformed bound constraint (6) is of the joint-velocity level (i.e., to limit $\dot{\theta}$ according to the current value and safety margins of θ). The physical meaning (and/or design mechanism) of the transformation can be explained by taking the right-hand inequality of (6) as an example. As joint variable θ increases (towards its upper bound θ^+), evidently the upper bound of joint-velocity variable $\dot{\theta}$ should accordingly decrease. By using $\mu(\theta^+ - \theta)$ as the right-hand side of (6), we can achieve so readily. More importantly, if θ reaches its upper limit θ^+ (i.e., if $\theta = \theta^+$), the right-hand inequality of (6) becomes $\dot{\theta} \leq \mu(\theta^+ - \theta) = 0$ which means that joint variable θ can not increase any more (either staying at θ^+ or returning towards θ^-). Thus, the joint variable θ will never exceed its upper limit θ^+ . Similarly, joint variable θ will not exceed its lower limit θ^- either, owing to the left-hand inequality of bound constraint (6).

Eqs. (5) and (6) can thus be combined into one dynamic bound-constraint $\xi^- \leq \dot{\theta} \leq \xi^+$ in a unified manner, where the i th elements of ξ^- and ξ^+ are defined respectively as

$$\xi_i^- = \max\{\dot{\theta}_i^-, \mu(\theta_i^- - \theta_i)\}, \quad \xi_i^+ = \min\{\dot{\theta}_i^+, \mu(\theta_i^+ - \theta_i)\}.$$

It follows from Section 2.1 and the above conversion that the physically-constrained drift-free redundancy-resolution problem of robot manipulators can be reformulated finally as the following quadratic program:

$$\text{minimize} \quad \frac{1}{2} \dot{\theta}^T W \dot{\theta} + z^T \dot{\theta}, \quad (7)$$

$$\text{subject to} \quad J \dot{\theta} = b, \quad (8)$$

$$\xi^- \leq \dot{\theta} \leq \xi^+, \quad (9)$$

where coefficients $W := I$, $z := \lambda(\theta - \theta(0))$, $b := \dot{r}$, and $\xi^\pm$ are defined before.

3. Three neural-network QP-solvers

In this section, we present three kinds of recurrent neural networks for the online solution of quadratic program (7)–(9). Namely, a dual neural network (DNN), an LVI-based primal-dual neural network (LVI-PDNN), and a simplified LVI-based primal-dual neural network (S-LVI-PDNN). All the three kinds of neural networks have piecewise-linear dynamics and are thus easier to implement via hardware circuits. Moreover, they all could globally exponentially converge to the optimal solution of the above-presented strictly-convex quadratic-program (7)–(9).

3.1. Dual neural network

In this subsection, we present a dual-neural-network based QP-solver for the real-time drift-free redundancy resolution of physically-constrained manipulators. Its design procedure is presented as follows.

Firstly, the quadratic program (7)–(9) can be rewritten as

$$\text{minimize} \quad \frac{1}{2} \dot{\theta}^T W \dot{\theta} + z^T \dot{\theta} \quad (10)$$

$$\text{subject to} \quad \gamma^- \leq E \dot{\theta} \leq \gamma^+,$$

where $\gamma^- := [\dot{r}^T, (\xi^-)^T]^T$, $\gamma^+ := [\dot{r}^T, (\xi^+)^T]^T$ in this dual-neural-network design, and with I denoting the identity matrix, matrix E is defined as

$$E = \begin{bmatrix} J \\ I \end{bmatrix} \in R^{(n+m) \times n}.$$

Secondly, at some time instant t , the QP problem depicted in (7)–(9) could be viewed as a parametric optimization problem. Now, following this line of reasoning, let us analyze the optimal solution to it and its equivalent problem-formulation (10). By the Karush–Kuhn–Tucker condition [23], we know that $\dot{\theta}$ is a solution to (10) if and only if there is a dual-decision-variable-vector $u \in R^{m+n}$ such that $\dot{\theta} - E^T u + z = 0$ (in view of $W = I = W^{-1}$) and

$$\begin{cases} [E \dot{\theta}]_i = \gamma_i^-, & \text{if } u_i > 0, \\ \gamma_i^- \leq [E \dot{\theta}]_i \leq \gamma_i^+, & \text{if } u_i = 0, \\ [E \dot{\theta}]_i = \gamma_i^+, & \text{if } u_i < 0. \end{cases} \quad (11)$$

In addition, condition (11) is equivalent to the system of piecewise linear equations $E \dot{\theta} = P_\Omega(E \dot{\theta} - u)$ (see [18] and references therein), where $P_\Omega(\cdot)$ is a projection operator from R^{n+m} onto $\Omega := \{u | \gamma^- \leq u \leq \gamma^+\} \subset R^{n+m}$, and the i th output of $P_\Omega(u)$ is defined as

$$\begin{cases} \gamma_i^-, & \text{if } u_i < \gamma_i^- \\ u_i, & \text{if } \gamma_i^- \leq u_i \leq \gamma_i^+ \\ \gamma_i^+, & \text{if } u_i > \gamma_i^+ \end{cases}, \quad \forall i \in \{1, 2, \dots, n+m\}.$$

Thirdly, by the above analysis, the necessary and sufficient condition for solving QP (10) is that $\dot{\theta}$ and u satisfy $\dot{\theta} - E^T u + z = 0$ and $E \dot{\theta} = P_\Omega(E \dot{\theta} - u)$. Substituting the former equation into the latter, we have

$$\begin{cases} \dot{\theta} = E^T u - z, \\ P_\Omega(E E^T u - E z - u) = E E^T u - E z, \end{cases}$$

which gives rise to the dynamic equation of dual neural network solving QP (10) as well as QP (7)–(9):

$$\dot{u} = c (P_\Omega(E E^T u - E z - u) - E E^T u + E z), \quad (12)$$

$$\dot{\theta} = E^T u - z, \quad (13)$$

where $c > 0$ is a design parameter used to scale the convergence rate of neural networks. Furthermore, we could have the following important results [18].

Theorem 1. Assume the existence of optimal solution $\dot{\theta}^*$ to strictly-convex QP problem (7)–(10) (in view of $W = I$). Starting from any initial state $u(0)$, dual neural network (12) is convergent to an equilibrium point u^* , of which the neural-network output $\dot{\theta}^* = E^T u^* - z$ is the optimal solution to QP (10) as well as the inverse-kinematic QP problem (7)–(9). Moreover, if there exists a constant $\rho > 0$ such that $\|P_\Omega(E E^T u - E z - u) - (E E^T u - E z)\|_2^2 \geq \rho \|u - u^*\|_2^2$, then the exponential convergence can be achieved for such a dual neural network. The exponential convergence rate is proportional to $c\rho$.

3.2. LVI-based primal-dual neural networks

In this subsection, by using a different design method, we develop an LVI-based primal-dual neural network (the original model) and its simplified model both for solving the inverse-kinematics QP problem (7)–(9). Their design procedure is as follows.

Firstly, different from Section 3.1, we can convert QP problem (7)–(9) alternatively to a set of linear variational inequalities. That is, to find a primal-dual equilibrium vector $v^* \in \Omega := \{u | \gamma^- \leq u \leq \gamma^+\} \subset R^{n+m}$ such that

$$(v - v^*)^T (Mv + q) \geq 0, \quad \forall v \in \Omega, \quad (14)$$

where the primal-dual decision variable vector v and its upper/lower bounds $\gamma^\pm$ (in this LVI-based primal-dual neural-network design) are defined respectively as (with $\varpi \gg 0 \in R^m$ to replace the m -dimensional $+\infty$ numerically):

$$v = \begin{bmatrix} \theta \\ y \end{bmatrix}, \quad \gamma^+ = \begin{bmatrix} \xi^+ \\ \varpi \end{bmatrix}, \quad \gamma^- = \begin{bmatrix} \xi^- \\ -\varpi \end{bmatrix} \in R^{m+n}.$$

In the above definition of v , its constituting vector $y \in R^m$ is a dual decision variable vector defined for equality constraint (8). In (14), coefficient matrix M and vector q are defined as follows by augmentation:

$$M = \begin{bmatrix} W & -J^T \\ J & 0 \end{bmatrix}, \quad q = \begin{bmatrix} z \\ -b \end{bmatrix} \in R^{n+m}.$$

In summary, we could have the following statement of this design step [19,20].

Theorem 2. By assuming the existence of optimal solution $\dot{\theta}^*$ (in view of $W = I$), the QP problem (7)–(9) could be converted to the LVI problem (14).

Secondly, it is known that LVI problem (14) is equivalent to the system of piecewise-linear equations [23,24,19–21],

$$P_\Omega (v - (Mv + q)) - v = 0, \quad (15)$$

where $P_\Omega(\cdot)$ is a projection operator from space R^{n+m} onto set Ω , which are defined to be the same as in Section 3.1 (except for γ^+ and γ^-).

Thirdly, to solve LVI (15) as well as QP (7)–(9), guided by dynamic-system-solver design experience [25,22,18], we could adopt the following dynamic equation of so-called LVI-based primal-dual neural network:

$$\dot{v} = c (I + M^T) (P_\Omega (v - (Mv + q)) - v) \quad (16)$$

where design parameter $c > 0$ is defined to be the same as for dual neural network (12). Furthermore, we have the following important results [19,20].

Theorem 3. Assume the existence of optimal solution $\dot{\theta}^*$ to QP (7)–(9) (in view of $W = I$). Starting from any initial state $v(0)$, LVI-based primal-dual neural network (16) is convergent to an equilibrium point v^* , of which the first n elements constitute the optimal solution $\dot{\theta}^*$ to the inverse-kinematic QP problem (7)–(9). Moreover, if there exists a constant $\rho > 0$ such that $\|v - P_\Omega (v - (Mv + q))\|_2^2 \geq \rho \|v - v^*\|_2^2$, then the exponential convergence can be achieved for such an LVI-based primal-dual neural network. The exponential convergence rate is proportional to $c\rho$.

By removing the scaling term $(I + M^T)$ of LVI-based primal-dual neural network (16) (i.e., the original model), we could have its simplified model (i.e., a so-called “simplified LVI-based primal-dual neural network”) as below:

$$\dot{v} = c (P_\Omega (v - (Mv + q)) - v). \quad (17)$$

In addition, we have the following important results [26].

Theorem 4. Assume the existence of optimal solution $\dot{\theta}^*$ to strictly-convex QP problem (7)–(9) (in view of $W = I$). Being the first n elements of neural state $v(t)$, the output x of simplified LVI-based primal-dual neural network (17) is globally exponentially convergent to optimal solution $\dot{\theta}^*$. In addition, the exponential convergence rate is proportional to c .

At the end of this section, it is worth presenting the comparison results about the computational cost of the above three neural-network-solvers and conventional numerical algorithms in the following two remarks.

Remark 2. Generally speaking, for the solution of the quadratic programs, numerical algorithms [23,9,27,28] performed on digital computers (i.e., on our today’s computers) could be employed. However, the minimal arithmetic operations of a QP numerical algorithm are usually proportional to the cube of the decision-variable-vector’s dimension (i.e., of expensively $O(n^3)$ operations) [23,9,27,28], and consequently such a numerical algorithm may not be efficient enough for large-scale real-time applications. In contrast, many parallel-processing computational methods simulated and/or implemented on specific architectures, e.g., the neural-dynamic and analog solvers [29,25,15,14,17,20,26,21,22,18], have also been investigated and could be applied to solving the online quadratic-programming problem. Such a neural-dynamic approach has now been regarded as a powerful alternative for real-time computation in view of its parallel-processing distributed nature and convenience of hardware implementation (see [10,29,13,17,18] and the references therein). Moreover, if we exploit the MATLAB numerical routine “QUADPROG” to solve the time-varying QP problem (7)–(9), the syntax is $\dot{\theta} = \text{QUADPROG}(I, z, [], [], J, \dot{r}, \xi^-, \xi^+)$ [28], which seemingly does not make full use of the iteratively time-varying nature of QP (7)–(9) (i.e., coefficients $z, J, \dot{r}, \xi^-$ and ξ^+ are time-varying). In contrast, corresponding to such iterative changes (usually being slight at every step) of the QP problem (7)–(9), the presented neural-network-solvers all have the self-tuning and self-adaptation properties (see the dynamic equations of the neural networks), which could take advantage of the time-varying nature of the QP problem (7)–(9).

Remark 3. The computational cost of the conventional dual neural network solving general QP problems is theoretically the highest among the presented three neural-network models, due to its matrix-inversion requirement (which is of expensively $O(n^3)$ operations). In addition, it can be calculated that the dual neural network (12) needs to perform $10n + 4m$ additions/subtractions, $5n(n + m)$ multiplications, $m + n$ integrator-operations and $m + n$ limiter-operations per iteration. In contrast, for the same QP-solving and redundancy-resolution purposes, the LVI-based primal-dual neural network (16) performs $6(n + m)$ additions/subtractions, $2(n + m)^2$ multiplications, $m + n$ integrator-operations and $m + n$ limiter-operations per iteration. As compared to the above two neural-network solvers, the simplified LVI-based primal-dual neural network (17) has theoretically simpler computational complexity, i.e., performing $4(n + m)$ additions/subtractions, $(n + m)^2$ multiplications, $m + n$ integrator-operations and $m + n$ limiter-operations per iteration. With $n = 4$ and $m = 2$ in our simulation, the simplified LVI-based primal-dual neural network (17) has evidently the lowest structural and computational complexity among the three neural solvers investigated in this paper.

4. Simulation studies

In this section, we apply the above three kinds of neural networks to the drift-free redundancy resolution of four-link planar robot manipulator, which has four degrees of freedom. Computer simulation has been performed for the straight-line path-following task of the manipulator end-effector. In this study, only the end-effector positioning is considered, the Jacobian matrix is 2×4 , and the degrees of redundancy is thus 2. Before starting this section, it is worth giving more simulation

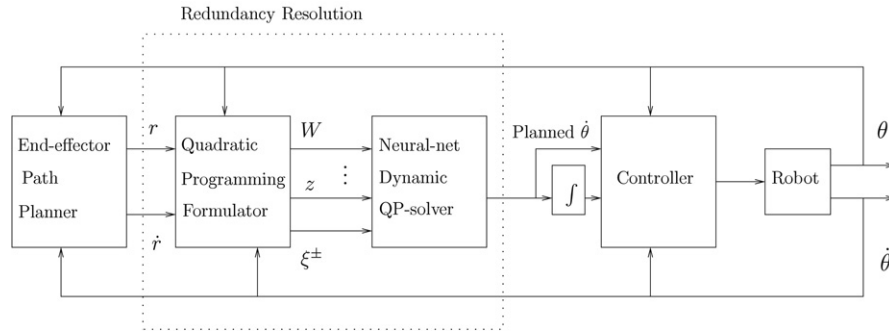


Fig. 1. Block diagram of QP-based neural-dynamic redundancy-resolution system.

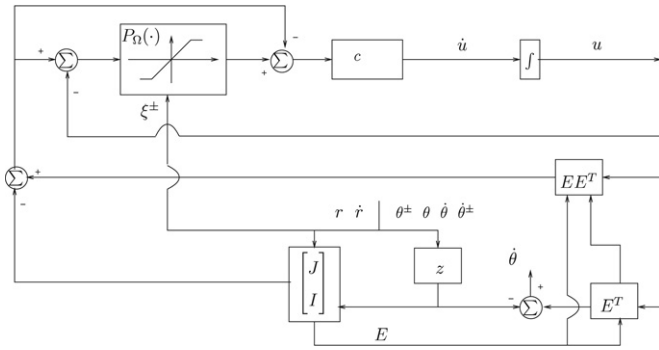


Fig. 2. Block diagram of dual neural network (12) and (13) solving QP (7)–(9).

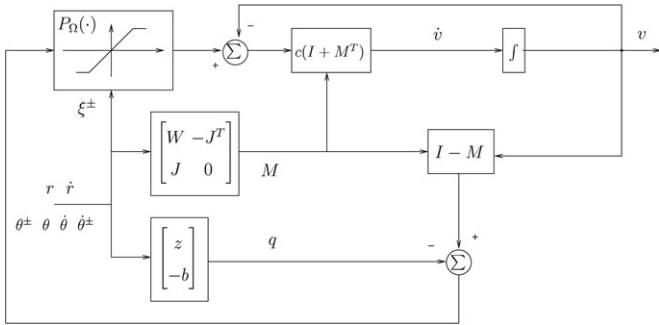


Fig. 3. Block diagram of LVI-based primal-dual neural network (16) solving (7)–(9).

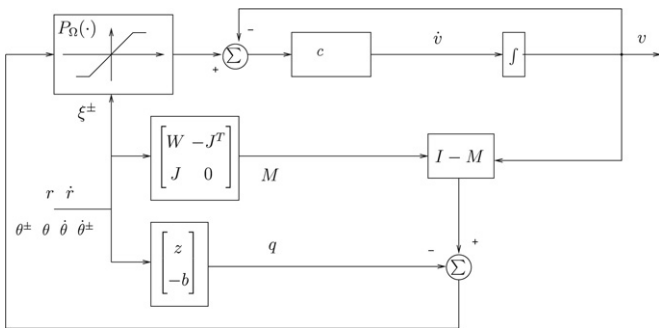


Fig. 4. Block diagram of simplified LVI-based primal-dual neural network (17).

and implementation details about the QP-based neural-dynamic redundancy-resolution scheme and system, of which the block diagrams are shown in Figs. 1–4.

In the computer simulation, the end-effector of such a four-link planar robot arm is expected to move forwards and backwards along a straight-line segment, like a repetitive spot-welding robot. The straight-line segment of length 0.7958 m, at every motion cycle, starts from initial state $\theta(0) = [3\pi/4, -\pi/2, -\pi/4, 0]^T$ rad,

Table 1

Joint displacement when drift-free criterion is not considered.

Joint #	$\theta(0)$ in radians	$\theta(20)$ in radians	$\theta(20) - \theta(0)$
θ_1	+2.35619449019234	+2.17015462263822	−0.186039867554
θ_2	−1.57079632679490	−1.21951682522814	+0.351279501566
θ_3	−0.78539816339745	−0.99814131436486	−0.212743150967
θ_4	+0.00000000000000	−0.13122343656764	−0.131223436567

Table 2

Joint displacement when drift-free criterion is considered.

Joint #	$\theta(0)$ in radians	$\theta(20)$ in radians	$\theta(20) - \theta(0)$
θ_1	+2.35619449019234	+2.35619604712185	$+1.55692 \times 10^{-6}$
θ_2	−1.57079632679490	−1.57079604572968	$+0.28106 \times 10^{-6}$
θ_3	−0.78539816339745	−0.78539901641950	$−0.85302 \times 10^{-6}$
θ_4	+0.00000000000000	−0.00000050601512	$−0.50601 \times 10^{-6}$

and shall return to such an initial state finally. The duration of the path-following task at every motion cycle is 20 s, and we limit the joint vector and joint velocity vector respectively as $\pm[4\pi/5, 4\pi/5, 4\pi/5, 4\pi/5]^T$ rad and $\pm[\pi, \pi, \pi, \pi]^T$ rad/s.

Firstly, we show the redundancy-resolution results synthesized by using the LVI-based primal-dual neural network (16) but without considering drift-free criterion [i.e., $\lambda = 0$ and $z = 0$ in (7)]. Design parameter $c = 10^8$ is taken. Fig. 5 shows that although all joint variables have been kept within their physical limits, the final state of the four-link planar manipulator does not return back to its initial state (after the end-effector completes the straight-line path-following task). The joint displacement is shown in Table 1, which is of order 0.1 rad. Evidently, such a solution is not repetitive.

Secondly, for comparison, let us show the favorable effect of drift-free criterion (7) (e.g., by setting $\lambda = 4$). The LVI-based primal-dual neural network (16) is applied again to the four-link planar manipulator with joint physical limits considered. Design parameters are selected to be the same as the first simulation example except $\lambda = 4$ here. As shown in Fig. 6, the final state of the four-link planar manipulator coincides with its initial state very well. The joint displacement is shown in Table 2, which is of order 10^{-6} rad. Evidently, such a redundancy resolution is repetitive (or termed, drift-free) in the sense of engineering applications. It is worth further pointing out that such a joint displacement could theoretically be decreased to zero by increasing design parameters c and λ , which is numerically nonzero because the simulation has to be performed on finite-arithmetic finite-memory digital computers.

In summary, as shown by the above simulation examples, we could apply effectively the quadratic drift-free criterion (7), inverse-kinematic QP scheme formulation (7)–(9), recurrent neural networks depicted in (12), (16) and (17) to solving the joint angle drift problem of robot manipulators. Similar computer-simulation results have been achieved as well by using dual neural network (12) and simplified LVI-based primal-dual neural network

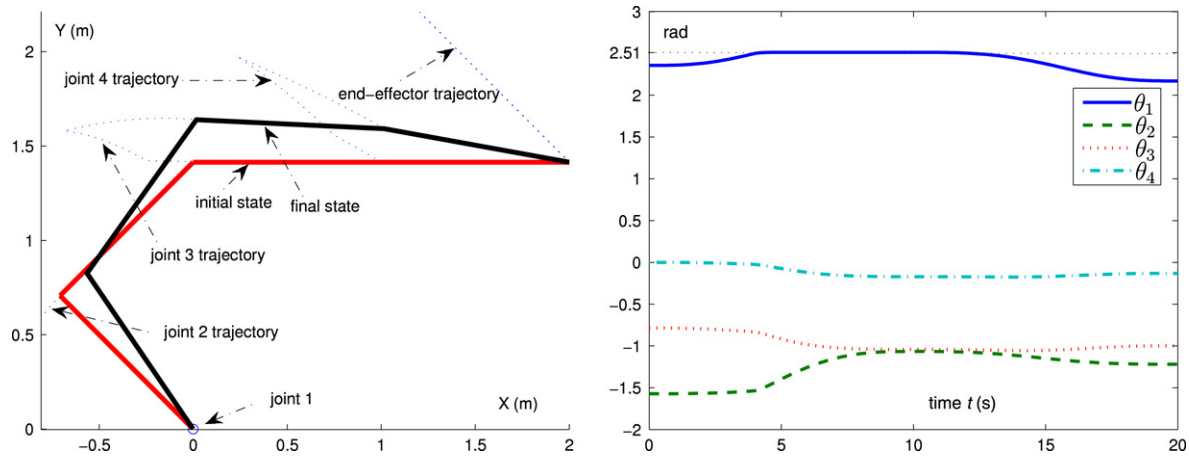


Fig. 5. Initial and final states are not equal (as drift-free criterion is not considered).

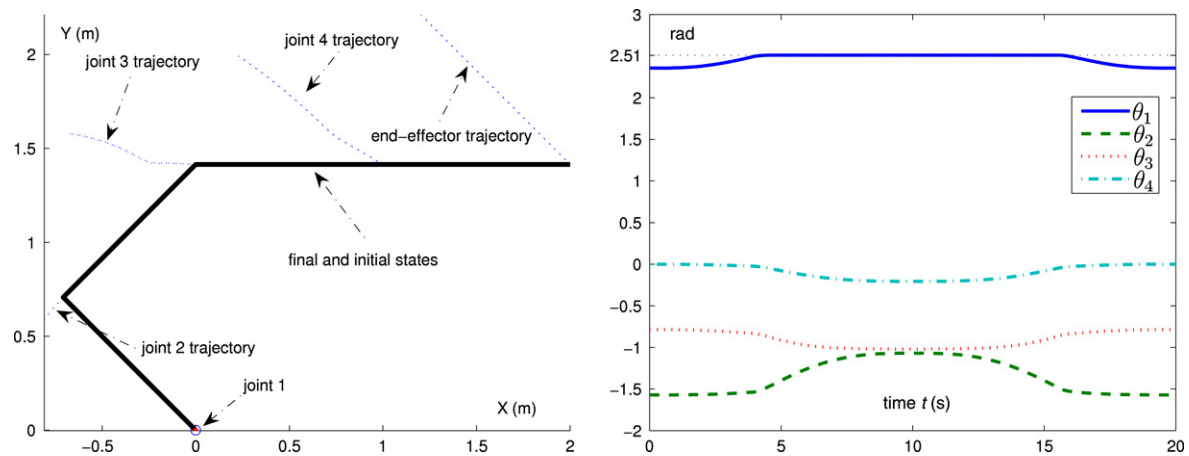


Fig. 6. Initial and final states are equal (as drift-free criterion is considered).

Table 3

Comparison on joint displacement synthesized by three kinds of neural networks.

Joint displacement	DNN (12)	LVI-PDNN (16)	S-LVI-PDNN (17)
$\theta_1(20) - \theta_1(0)$	$+3.57131 \times 10^{-7}$	$+1.55692 \times 10^{-6}$	$+9.75728 \times 10^{-7}$
$\theta_2(20) - \theta_2(0)$	$+1.25605 \times 10^{-7}$	$+0.28106 \times 10^{-6}$	$+2.36738 \times 10^{-7}$
$\theta_3(20) - \theta_3(0)$	-3.32220×10^{-7}	-0.85302×10^{-6}	-5.61153×10^{-7}
$\theta_4(20) - \theta_4(0)$	-2.45591×10^{-7}	-0.50601×10^{-6}	-3.59248×10^{-7}

Table 4

Comparison on simulation runtime synthesized by three kinds of neural networks.

Test #	By DNN (12)	By LVI-PDNN (16)	By S-LVI-PDNN (17)
1	1.04856	1.87816	5.61296
2	1.05934	1.85672	5.71167
3	1.03853	1.87721	5.68398
4	1.06568	1.91157	5.62340
5	1.07239	1.91076	5.62898
6	1.08298	1.89559	5.84173
7	1.06786	1.88669	5.70509
8	1.03163	1.86975	5.68382
9	1.04111	1.88560	5.93385
10	1.05546	1.91103	5.65361

(17). Because of their figures' similarity to those of LVI-based primal-dual neural network (16), we do not show them here but would like to supply upon request. For comparative purposes, Tables 3 and 4 are provided, which elaborates on the robot joint-displacement and simulation-runtime values (respectively, in radians and in seconds) synthesized by the presented RMP

scheme (7)–(9) and the three kinds of neural-network-solvers. The simulation and comparison results substantiate well the efficacy of all of the three neural networks which act as the real-time solver of the presented RMP scheme for redundant robot manipulators.

5. Concluding remarks

In this paper, as illustrated based on a four-link planar robot manipulator, we have basically solved the joint angle drift problem of physically constrained redundant manipulators. The solution, based on a QP problem formulation, could be synthesized by three kinds of neural network solvers. The neural-network architectural and computational complexities are simpler than other existing recurrent neural networks', in addition to their piecewise-linear dynamics. These neural networks are also shown to be globally exponentially convergent to optimal solutions of such a strictly-convex quadratic program used in the repetitive redundancy-resolution of robot manipulators. Computer-simulation results based on the four-link robot have demonstrated the efficacy of drift-free scheme formulation (7)–(9) and its neural-network solvers on real-time kinematic control of joint-constrained redundant manipulators.

Acknowledgments

This research is funded by National Science Foundation of China under Grant 60775050 and by the Program for New Century

Excellent Talents in University (NCET-07-0887). The authors would also like to thank the editors and anonymous reviewers for their time and effort in providing many constructive and inspiring comments. In addition, the gratitude is expressed to the staff working at the Science and Technology Office of Sun Yat-sen University as well for their generous support and advices in many aspects of research.

References

- [1] L. Sciavicco, B. Siciliano, Modeling and Control of Robot Manipulators, Springer-Verlag, London, UK, 2000.
- [2] M. Galicki, Path-constrained control of a redundant manipulator in a task space, *Robotics and Autonomous Systems* 54 (3) (2006) 234–243.
- [3] E.S. Conkur, Path following algorithm for highly redundant manipulators, *Robotics and Autonomous Systems* 45 (1) (2003) 1–22.
- [4] O. Khatib, A. Bowling, Optimization of the inertial and acceleration characteristics of manipulators, in: *Proc. IEEE Int. Conf. Robot. Automat.* vol. 4, 1996, pp. 2883–2889.
- [5] A. Liegeois, Automatic supervisory control of the configuration and behavior of multibody mechanisms, *IEEE Transactions on Systems, Man and Cybernetics* 7 (12) (1997) 868–871.
- [6] F.T. Cheng, T.H. Chen, Y.Y. Sun, Resolving manipulator redundancy under inequality constraints, *IEEE Journal of Robotics and Automation* 10 (1) (1994) 65–71.
- [7] M. Vukobratovic, M. Kircanski, A dynamic approach to nominal trajectory synthesis for redundant manipulator, *IEEE Transactions on Systems, Man and Cybernetics* 14 (4) (1984) 580–586.
- [8] T.M. Abdel-Rahman, A generalized practical method for analytic solution of the constrained inverse kinematics problem of redundant manipulators, *International Journal of Robotics Research* 10 (4) (1991) 382–395.
- [9] F.T. Cheng, R.J. Sheu, T.H. Chen, The improved compact QP method for resolving manipulator redundancy, *IEEE Transactions on Systems, Man and Cybernetics* 25 (11) (1995) 1521–1530.
- [10] H. Ding, J. Wang, Recurrent neural networks for minimum infinity-norm kinematic control of redundant manipulators, *IEEE Transactions on Systems, Man and Cybernetics A* 29 (3) (1999) 269–276.
- [11] Z. Mao, T.C. Hsia, Obstacle avoidance inverse kinematics solution of redundant robots by neural networks, *Robotica* 15 (1) (1997) 3–10.
- [12] W.S. Tang, J. Wang, A recurrent neural network for minimum infinity-norm kinematic control of redundant manipulators with an improved problem formulation and reduced architectural complexity, *IEEE Transactions on Systems, Man and Cybernetics B* 31 (1) (2001) 98–105.
- [13] J. Wang, Recurrent neural networks for computing pseudoinverses of rank-deficient matrices, *SIAM Journal on Scientific Computing* 18 (5) (1997) 1479–1493.
- [14] Y. Xia, A new neural network for solving linear programming problems and its application, *IEEE Transactions on Neural Networks* 7 (3) (1996) 525–529.
- [15] J. Wang, Y. Zhang, Recurrent neural networks for real-time computation of inverse kinematics of redundant manipulators, in: P. Sincak, J. Vascak, K. Hirota (Eds.), *Machine Intelligence: Quo Vadis*, World Scientific, Singapore, 2004.
- [16] C. Klein, C. Huang, Review of pseudoinverse control for use with kinematically redundant manipulators, *IEEE Transactions on Systems, Man and Cybernetics* 13 (1983) 245–250.
- [17] Y. Xia, J. Wang, A dual neural network for kinematic control of redundant robot manipulators, *IEEE Transactions on Systems, Man and Cybernetics B* 31 (1) (2001) 147–154.
- [18] Y. Zhang, J. Wang, A dual neural network for convex quadratic programming subject to linear equation and inequality constraints, *Physics Letters A* 298 (4) (2002) 271–278.
- [19] Y. Zhang, Inverse-free computation for infinity-norm torque minimization of robot manipulators, *Mechatronics* 16 (2006) 177–184.
- [20] Y. Zhang, S.S. Ge, T.H. Lee, A unified quadratic programming based dynamical system approach to joint torque optimization of physically constrained redundant manipulators, *IEEE Transactions on Systems, Man and Cybernetics B* 34 (5) (2004) 2126–2132.
- [21] Y. Zhang, X. Lv, Z. Yang, Z. Li, Repetitive motion planning of redundant robots based on LVI-based primal–dual neural network and PUMA560 example, *Lecture Notes in Bioinformatics* 4689 (2007) 536–545.
- [22] Y. Zhang, J. Wang, A dual neural network for constrained joint torque optimization of kinematically redundant manipulators, *IEEE Transactions on Systems, Man and Cybernetics B* 32 (5) (2002) 654–662.
- [23] M.S. Bazaraa, H.D. Sherali, C.M. Shetty, *Nonlinear Programming Theory and Algorithms*, Wiley, New York, 1993.
- [24] M.C. Ferris, J.S. Pang, Complementarity and Variational Problems, in: *State of the Art*, SIAM, Philadelphia, 1997.
- [25] J. Wang, Q. Hu, D. Jiang, A Lagrangian network for kinematic control of redundant manipulators, *IEEE Transactions on Neural Networks* 10 (5) (1999) 1123–1132.
- [26] Y. Zhang, Z. Li, H.Z. Tan, Z. Fan, On the simplified LVI-based primal–dual neural network for solving LP and QP problems, in: *Proc. IEEE Int. Conf. Contr. Automat.* 2007, pp. 3129–3134.
- [27] W. Li, J. Swetits, A new algorithm for solving strictly convex quadratic programs, *SIAM Journal on Optimization* 7 (3) (1997) 595–619.
- [28] The MathWorks Inc., *Optimization Toolbox for Use with MATLAB*, version 2.3, 2003.
- [29] C. Mead, *Analog VLSI and Neural Systems*, Addison-Wesley, Reading, MA, 1989.



Yunong Zhang is a professor at Department of Electronics and Communication Engineering, Sun Yat-Sen University (SYSU), China. He received his B.S., M.S., and Ph.D. degrees respectively from Huazhong University of Science and Technology (HUST), South China University of Technology (SCUT), and Chinese University of Hong Kong (CUHK) and respectively in 1996, 1999, and 2003. Before joining SYSU in 2006, Yunong Zhang had been with National University of Ireland (NUI), University of Strathclyde, National University of Singapore (NUS) since 2003. His main research interests include robotics, neural networks,

and Gaussian processes.



Zhiguo Tan received his B.S. degree in electrical engineering and automation from Dalian Maritime University (DLMU), China in 2004. He is now pursuing his M.S. degree at Sun Yat-Sen University. His research interests include nonlinear systems, robotics, neural networks, virtual reality, and signal processing.



Ke Chen received his B.S. degree in automatic control engineering from Sun Yat-Sen University, China in 2007. He is now pursuing his M.S. degree at Sun Yat-Sen University. His research interests include nonlinear systems, robotics, neural networks, and signal processing.



Zhi Yang is a professor at Department of Electronics and Communication Engineering, Sun Yat-Sen University, China. He received his B.S. and M.S. degrees in automatic control from Gansu University of Technology, China, in 1983 and 1988, respectively. His research interests include automatic instruments, CIMS, industrial automation, and complex systems modeling.



Xuanjiao Lv received her B.S. degree in Electronic Information Science and Technology from Shanxi Datong University, China in 2006. She is now completing her M.S. degree at Sun Yat-Sen University. Her research interests include nonlinear systems, robotics, neural networks, and signal processing.